

De Script de Python a Web App en Segundos

```
1 def shout(text):  
2     return text.upper()  
3  
4 gr.Interface(fn=shout, inputs='text', outputs='text').launch()  
5 |
```

text

hola

Submit

output

HOLA

El Problema del Prototipado Rápido

“Como científicos de datos, queremos exponer nuestros modelos rápidamente, sin perder tiempo escribiendo HTML o CSS.”



El Reto: Crear aplicaciones con React o Node requiere código ‘boilerplate’ complejo.



La Solución: Gradio construye interfaces completas con una sola línea de Python.



La Promesa: Una herramienta diseñada para la alegría del desarrollador.

Los Cimientos: Importaciones y Seguridad

Code Card

La línea mágica:
Importa la
librería
completa.

```
import gradio as gr  
from dotenv import load_dotenv  
import os
```

```
load_dotenv(override=True)
```

Carga claves
API (.env) para
conectar con
OpenAI.

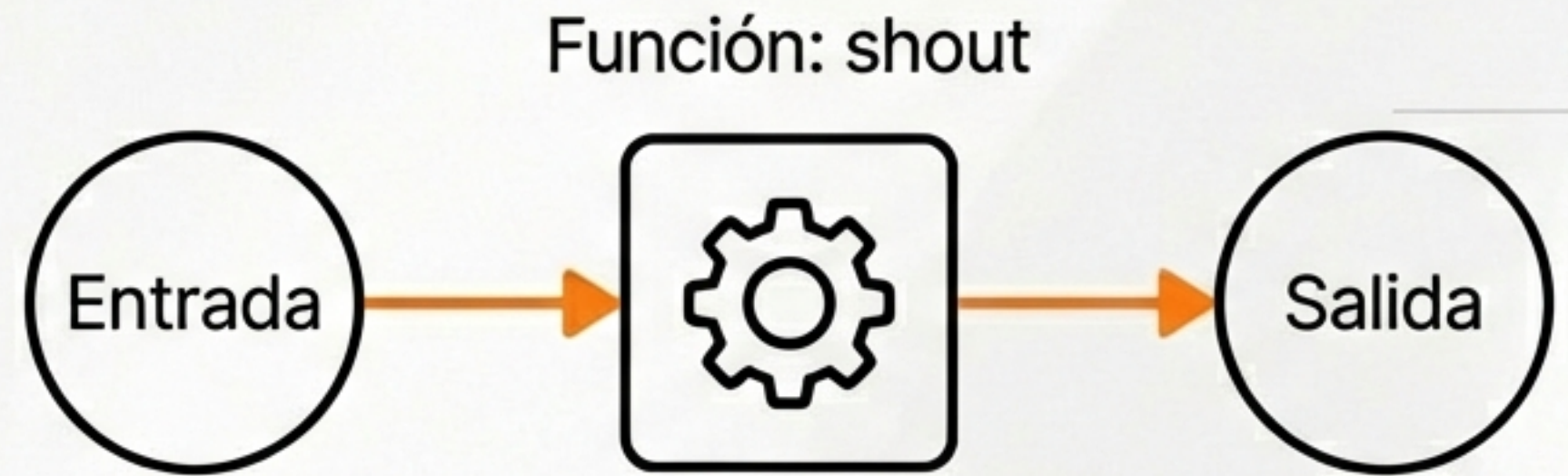
Sin estas llaves, el 'cerebro' de la IA no funcionará.

Paso 1: La Lógica del Backend

Creando el motor antes que el coche.

Code Card

```
def shout(text):  
    return text.upper()
```



'hola' → .upper() → 'HOLA'

Paso 2: Construyendo el Puente Visual

```
gr.Interface(fn=shout, inputs='textbox', outputs='textbox')
```

MOTOR:

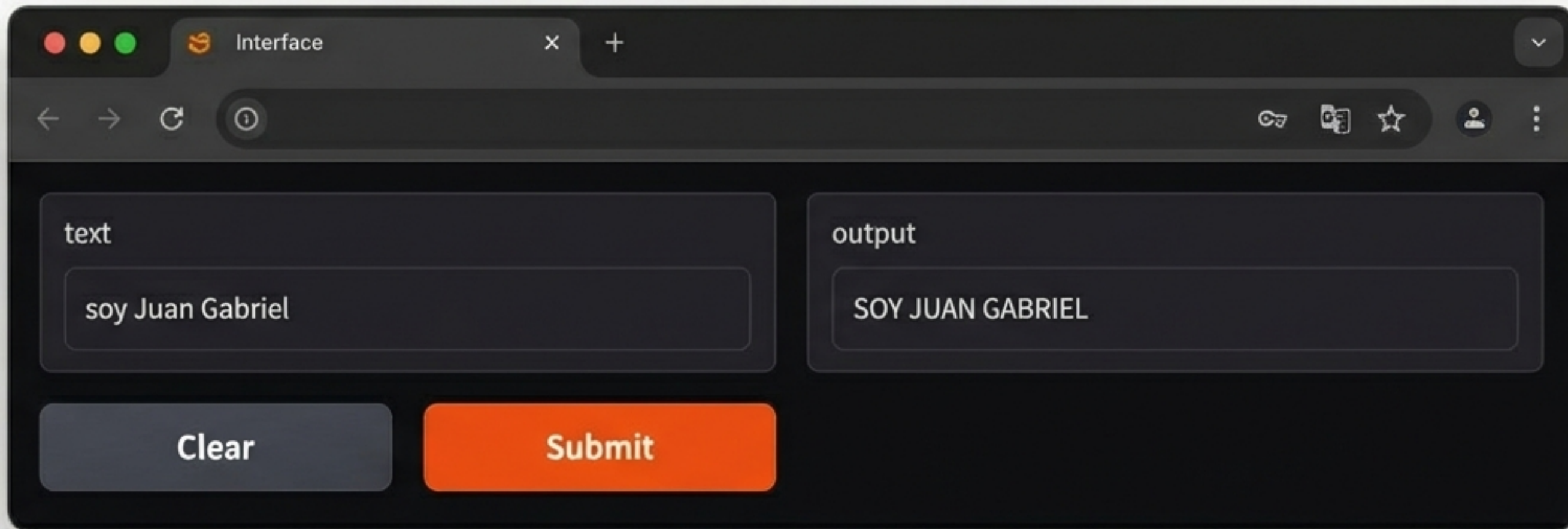
¿Qué lógica
ejecutamos?

ENTRADA:

¿Cómo recibimos
los datos?

SALIDA: ¿Dónde
mostramos el
resultado?

Resultado Visual: De Minúsculas a Mayúsculas



Input: 'soy Juan Gabriel' → Clic en Submit → Función shout()
→ Output: 'SOY JUAN GABRIEL'

Ejecución y Personalización

Lanzamiento Público

```
.launch(share=True)
```

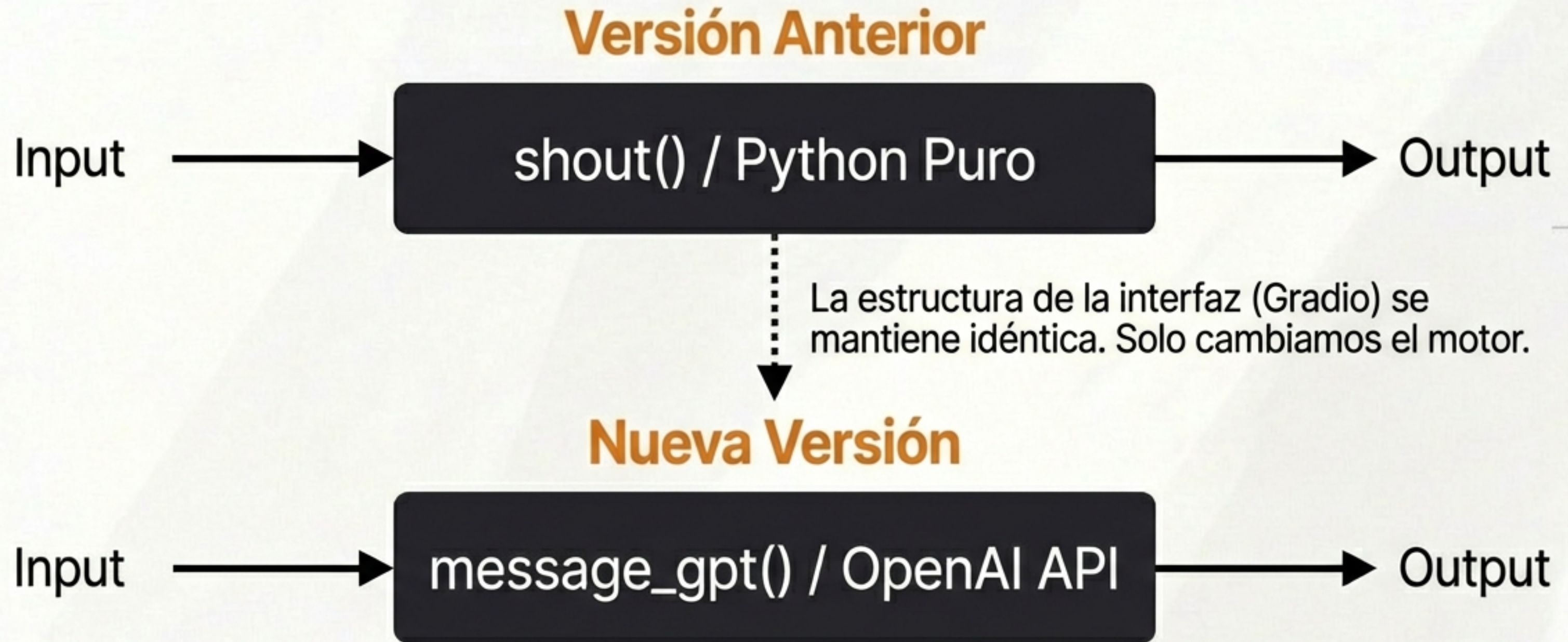
Genera un enlace público temporal (72h) que tuneliza hacia tu servidor local. Permite compartir tu app con el mundo.

Gestión de Datos

```
flagging_mode='never'
```

Desactiva el botón 'Flag'. Evita guardar datos de usuarios en carpetas locales (csv) para mantener la interfaz limpia.

Subiendo de Nivel: Integrando GPT

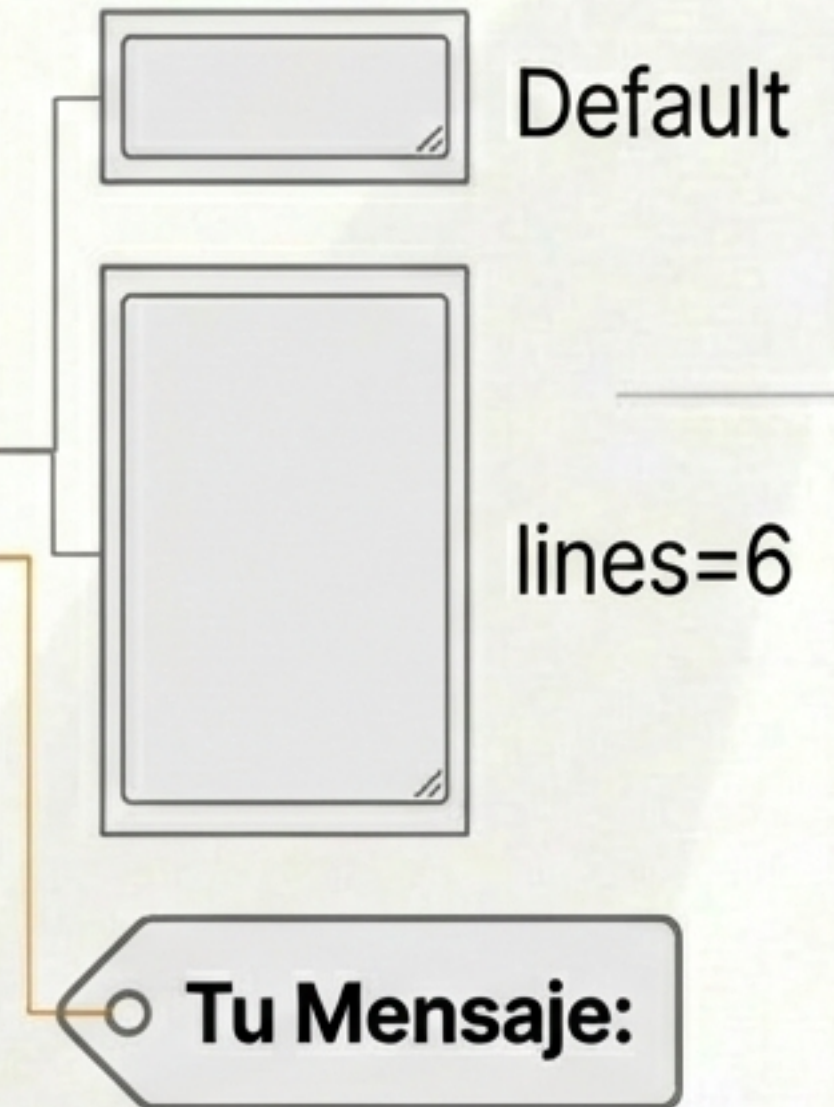


El Nuevo Motor: La Función message_gpt

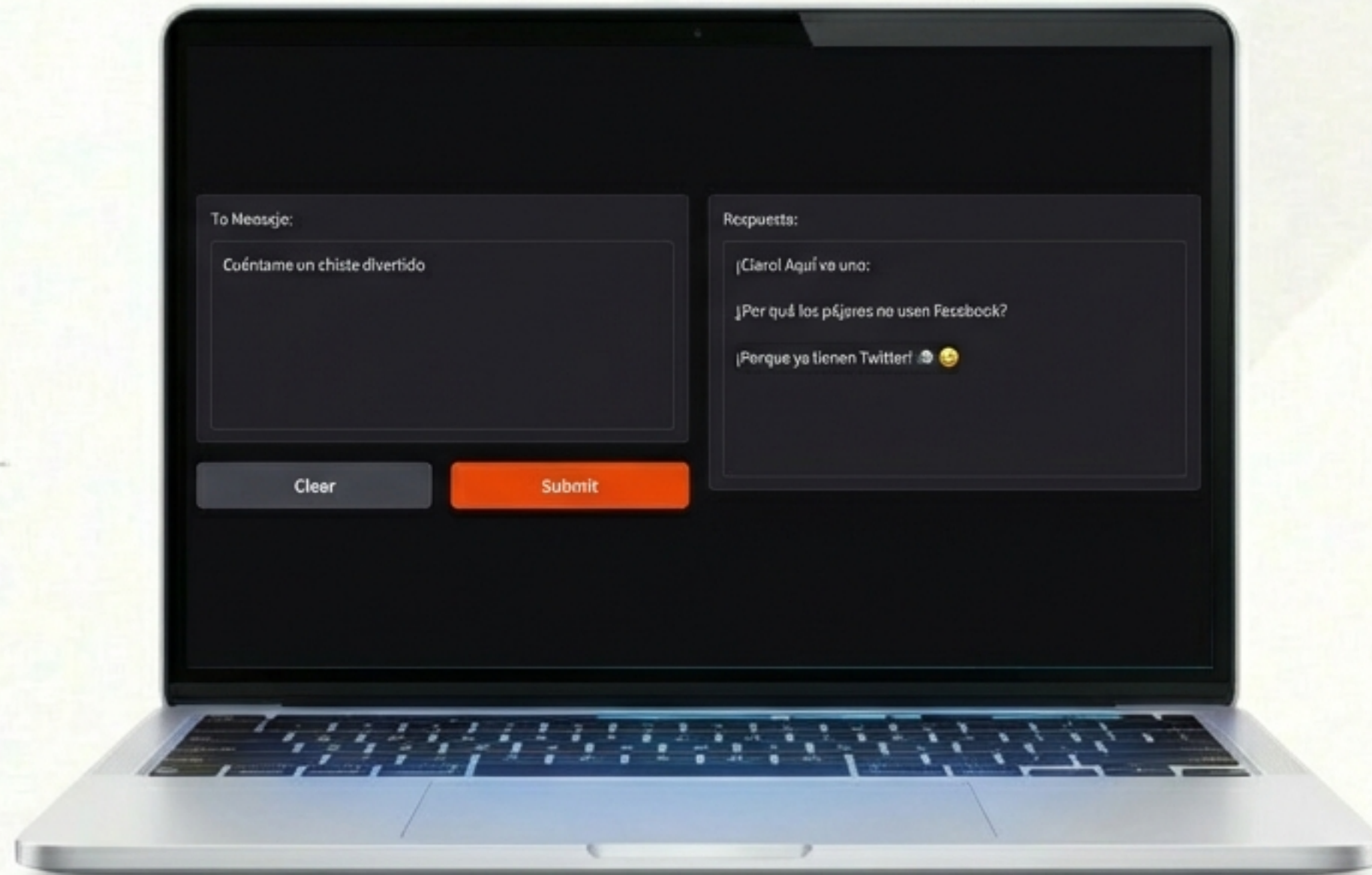
```
def message_gpt(prompt):  
    messages = [  
        {'role': 'system', 'content': 'You are a helpful assistant'},  
        {'role': 'user', 'content': prompt}  
    ]  
    completion = openai.chat.completions.create(  
        model='gpt-4o-mini', messages=messages  
    )  
    return completion.choices[0].message.content
```


Interfaz Gráfica para Chatbot

```
gr.Interface(  
    fn=message_gpt,  
    fn=message_gpt,  
    inputs=[gr.Textbox(label='Tu Mensaje:', lines=6)],  
    outputs=[gr.Textbox(label='Respuesta:', lines=8)],  
    flagging_mode='never'  
)
```



Resultado Final: Interactuando con GPT



Prompt: “Cuéntame un chiste divertido”

Respuesta: Generada por GPT-4o-mini y renderizada por Gradio.

Arquitectura: ¿Dónde ocurre la magia?



Usuario en Internet



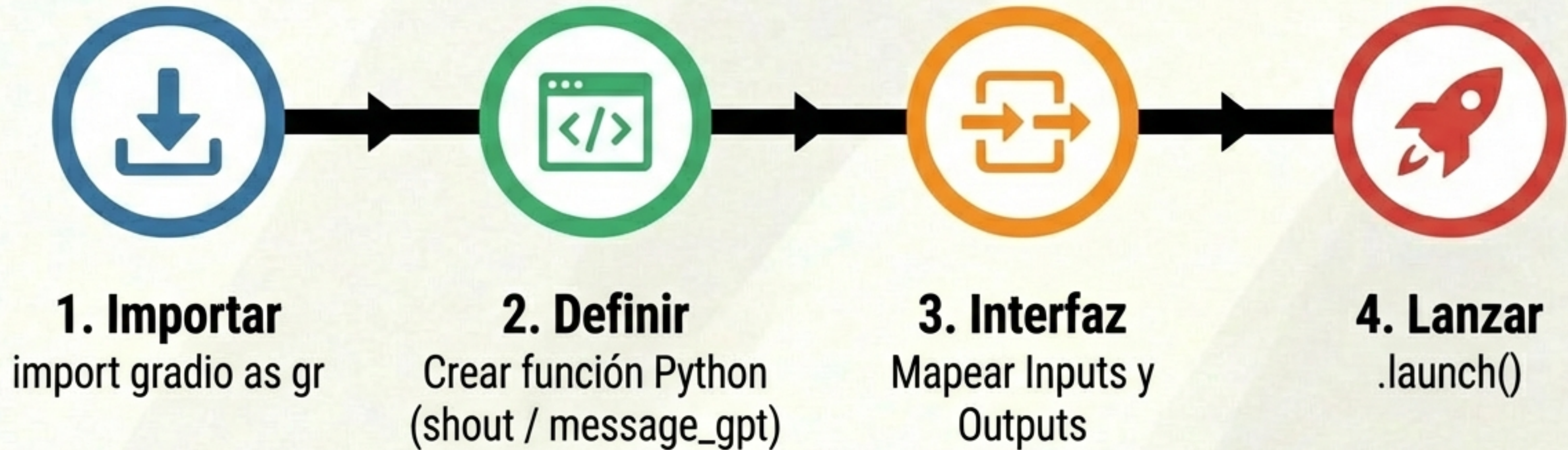
Túnel Seguro
(Public URL)



Tu Ordenador
(Localhost)

El código Python, tus claves API y el cómputo ocurren **AQUÍ**.
Gradio solo transmite el texto.

Resumen del Flujo de Trabajo



Gradio convierte scripts en aplicaciones.

Democratizando el acceso a la
Inteligencia Artificial.